Real-Time Fraud Detection Architecture for Digital Banking: A Systematic Literature Review Applied to the Moneda Segura Case

Juan José Rojas Correa, Nicolas Afonso Mejía, Nicholas Solis, Santiago Gómez Lopez, and Laura Andrea Cárdenas
Department of Software Engineering, Universidad Icesi, Cali, Colombia

Abstract: *This study focuses on developing a reliable, fast solution for real time fraud detection in banking systems. MonedaSegura is a banking application with 1.8 million active users that must detect and evaluate transaction fraud in under 500 ms, maintain a rich record of consumer behavior, and allow rapid deployment of new rules or models. To solve these challenges, the project conducted in-depth research on recommended architectures, using AI models to collect and perform an initial evaluation of relevant papers using the SLR method, screening 54 peer-reviewed sources through a quantitative Importance-Reliability Matrix. The study concluded that Apache Kafka with tiered storage is the best option for data ingestion; then, Apache Flink is the fastest for processing user records; and a Lambda pattern with a Heterogeneous Temporal Graph Neural Network provides the most modern and reliable solution for fraud detection, improving AUC-ROC from 0.851 to 0.940 over a tabular baseline while keeping inference below 2 ms. This model represents the ideal benchmark; however, the complexity of the implementation makes a Kappa-based architecture a viable alternative to the Lambda HTGNN pattern.*

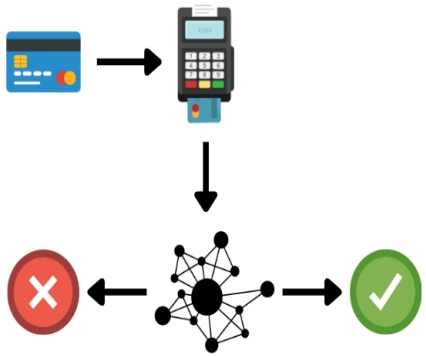
Index Terms: *fraud detection, stream processing, stateful computing, feature store, graph neural networks, MLOps, Lambda architecture, Kappa architecture.*

I. INTRODUCTION

Digital-only banks operate under a structural tension that traditional banking infrastructure never had to resolve, every transaction must be evaluated for fraud in real time. Yet the evidence needed to make that evaluation trustworthy (a user's accumulated behavioral history, device patterns, and relational context) grows richer the longer the system has been monitoring. MonedaSegura, the digital bank motivating this study, formalizes this tension into three concrete requirements set by its risk committee: transaction evaluation combining business rules and machine learning models within a sub-500-millisecond; features enriched with historical, per-user behavioral context rather than transaction-level data alone; a deployment process agile enough for the risk team to iterate on rules and models without multi-week release cycles.

These three requirements do not map onto a single engineering decision, and prior literature, discussed in Section I.A, tends to optimize only one of them at a time (streaming papers report latency without fraud accuracy, graph learning papers report accuracy without production cost, and MLOps papers govern reliability without reference to the architecture underneath it). None of them, taken alone, tells MonedaSegura what to build.

This paper closes that gap for MonedaSegura's specific case. We derive nine architectural decisions from a systematic literature review of 54 peer reviewed sources, spanning stateful stream processing, dynamic rescaling, cloud native messaging, heterogeneous temporal graph neural networks, feature stores, in network inference, MLOps robustness, and continuous experimentation governance. The decisions are organized across five system layers (ingestion, state, features, modeling, and governance), and for each one we report the evidence behind the option we chose, the alternative we discarded, and, where the evidence supports it, the conditions under which a smaller or more cost constrained bank should choose differently. Section I.A reviews related work, Sections II to IV present the nine decisions across the three architectural layers, Section V describes the SLR methodology, Section VI discusses trade-offs, Section VII defines the scope of the prototype implementation that will be built, and Section VIII concludes.



A. Related Work

Research supporting real-time fraud detection architectures has evolved along three separate tracks, each addressing a different layer of the problem without integrating the others.

Zhang et al. [23] survey transactional stream processing broadly, and Liu and Buyya [13] propose a taxonomy of resource management and scheduling motivated explicitly by latency-critical applications such as fraud detection. Both evaluate success in systems terms (throughput, latency, resource use), not in terms of fraud-detection accuracy.

A separate line of work targets the model itself. Dal Pozzolo et al. [4] address class imbalance and concept drift with tabular,

transaction-level features. More recent work moves to graph-based models: Xiang et al. [22] capture spatial and temporal patterns jointly, Wu et al. [21] apply heterogeneous graph neural networks to supply-chain finance fraud, and Cheng et al. [25] synthesize over one hundred such studies into a unifying framework. These report consistent accuracy gains over tabular baselines, but almost entirely on offline benchmarks, without reporting whether the resulting model meets a production latency budget such as MonedaSegura's sub-500-millisecond constraint.

A third line addresses post-deployment reliability. Gama et al. [8] established the foundational taxonomy of concept drift adaptation, Eken et al. [7] synthesize 150 studies into a catalogue of MLOps practices, and Bayram and Ahmed [1] show empirically that commercial MLOps platforms fail to adequately cover concept drift, generalization, and label noise out of the box.

II. INGESTION AND STATE LAYER

This section covers the system's data-entry and stateful-memory layer: how transactions are ingested without loss or duplication (Decision 3), how per-user behavioral history is kept in the streaming engine's state (Decision 1), and how the system absorbs traffic spikes without downtime (Decision 2).

A. Decision 1 Disaggregated State Management

Every behavioral feature that MonedaSegura's risk model consumes (transaction velocity, geographic displacement from the previous operation, device familiarity) requires the streaming engine to retain per-user history across events. At 1.8 million active users the working memory, known as state, must remain continuously available while the platform processes transactions without interruption. The decision here is where that state physically resides. We considered two options: the conventional design, in which the state store is embedded on the local disk of each compute node, and a disaggregated design, in which the primary state lives in remote storage with local disk acting only as a cache.

We chose the disaggregated design, following the architecture introduced in Apache Flink 2.0. State updates are streamed continuously to a remote distributed file system, and an asynchronous execution model compensates for the additional access latency that remote storage would otherwise introduce. The underlying principle, managing compute, memory and storage as independently scalable pools rather than aggregating them in monolithic servers, is not specific to stream processing: Wang and Zhang [20] report that disaggregated designs have been widely adopted by major cloud providers, including Amazon, Microsoft, Alibaba, Google and IBM.

This choice is based on Mei et al. [15], who evaluate the architecture against Flink 1.20 with RocksDB on local disk, using a scaled-down replication of a production logistics workload that yields roughly 290 GB of total state. Three of their results are relevant here. The first, and the most important for our case, is recovery time. Recovering a job holding 290 GB of state takes over three minutes under the baseline, and rescaling exceeds five minutes; the disaggregated architecture reports recovery after failure or rescaling up to 49 times faster. At a peak load of 500 transactions per second, a three-minute state reconstruction leaves on the order of 90,000 transactions unevaluated, which the risk committee's continuity requirement does not tolerate.

The second is checkpoint behavior, which improves not only on average but in its tail. Under the baseline, with an average incremental checkpoint size of 1.89 GB, more than 19.7% of checkpoints took longer than 30 seconds and more than 1.5% exceeded 50 seconds, because duration is largely determined by incremental checkpoint size. In the disaggregated architecture every checkpoint completed within three seconds regardless of size. Bounded checkpoint duration narrows the window of work that must be replayed after a failure and removes the dependence between state growth and snapshot cost.

The third is operating cost. Mei et al. [15] report that the baseline required at least 16 compute units, not because the workload demanded sixteen cores but because keeping per-unit state below the 20 GB disk limit that a compute unit bundles forced the allocation, while the disaggregated deployment sustained the same workload with 8, halving compute cost. The same coupling explains a broader finding in their production data: applying a threshold of 20 GB of state per compute unit, 35% of jobs in Alibaba's logistics business are disk-bound, requiring additional units solely to address storage limits rather than processing demand.

Three limitations temper this choice. The cost argument transfers only weakly to our case: MonedaSegura's behavioral state is estimated at 18 to 90 GB (1.8 million users at 10 to 50 KB per profile), which falls at or below the 20 GB-per-compute-unit threshold the study uses to classify jobs as disk-bound. At that scale storage is unlikely to be the binding constraint, and the reported 50% saving should not be expected to reproduce; we therefore adopt this architecture primarily for its recovery and checkpointing characteristics, which serve the continuity requirement directly, and treat cost efficiency as a secondary benefit that materializes only if state grows well beyond current projections. The architecture is also not free at runtime: Mei et al. [15] report a 30% average increase in CPU utilization for stateful operators, attributed to context-switching for asynchronous access, intra-task scheduling, batched I/O classification for remote access, and additional garbage collection for the futures the asynchronous model creates; this figure is measured on the Nexmark benchmark rather than a fraud-scoring workload, whose access pattern differs, so we treat it as an upper-bound planning input to validate against our own workload before committing capacity. Finally, the evidence rests on a single large-scale validation environment operated by one of the contributing organizations, although the work is co-

authored with KTH Royal Institute of Technology and Boston University rather than reported solely by that organization.

B. Decision 2 Fine-Grained Rescaling

Transaction volume at MonedaSegura is not constant: peak load reaches 500 transactions per second, and capacity must follow demand rather than being provisioned for the worst case at all times. Because the streaming operators that compute behavioral features are stateful and partitioned across workers, adding or removing a worker is not a matter of starting a process. It requires state migration: moving each user's accumulated history to the worker that will own it next. The decision here is how that migration is performed.

Gu et al. [9] classify prior approaches into four families. Full-Restart pauses and resumes the entire job while state is redistributed, taking the service offline precisely when load is highest. Partial-Pause restarts a subgraph rather than the whole topology, reducing but not eliminating blocking. Replicated-Dataflow runs a second dataflow in parallel with the original until migration completes, avoiding blocking at the cost of duplicated resources and additional de-duplication machinery. Proactive approaches, of which Megaphone [10] is the best known, add work during normal operation so that migration is cheaper when it eventually occurs, paying a permanent overhead for an occasional event. The authors identify a limitation shared by all four: they are order-unaware, meaning none considers the sequence in which state is migrated.

We adopt prioritized state migration, the mechanism Gu et al. [9] propose as an alternative to all four. State belonging to hot keys (those currently being processed or about to be processed downstream) is migrated first, and state is transferred at record granularity through a fetch-on-demand protocol that preserves consistency. The practical consequence is that users whose transactions are arriving right now regain a working feature path first, while migration of inactive users proceeds in the background. Unlike proactive designs, the mechanism imposes no work during non-rescaling periods; unlike restart-based designs, it does not block execution; and unlike replicated dataflows, it does not duplicate resources.

The evidence is a comparison against native Flink and an order-unaware baseline on a key-count job with 10^8 unique keys. Both baselines exhibit a latency peak three orders of magnitude above their normal operating level, decaying only gradually once the restart or migration completes. Prioritized migration holds latency under 600 ms throughout the migration stages and is the first of the three to return to its previous processing rate. Gu et al. [9] further contrast their design against two named systems: Megaphone [10] introduces partitioning overhead inside the processing operators during normal operation, and Rhino [6] incurs network communication even when no migration is taking place, whereas prioritized migration confines all its cost to the rescaling window itself.

Three qualifications matter for our case. First, 600 ms exceeds MonedaSegura's 500 ms scoring budget. Because inference runs inside the streaming job rather than outside it (Decisions 4 and 9), the two figures compete during a migration window. We do not claim that rescaling becomes invisible; we claim that its cost falls from tens of seconds to a few hundred milliseconds. This is also the exact condition Decision 7 anticipates: its circuit breaker triggers when the scoring service exceeds its latency budget, and reroutes the affected transactions to the last-known-good static rule set. The three decisions divide the problem between them: this one bounds how large the overshoot can be, Decision 9 explains why inference sits inside the streaming job, and Decision 7 defines what happens while it lasts.

Second, rescaling is an episodic event triggered by capacity changes, not a steady-state condition; the reported figure characterizes a transient window rather than normal operating latency. Third, the evaluation uses a synthetic key-count workload with roughly fifty times more keys than MonedaSegura has users, and without feature-computation logic in the path. The mechanism transfers, but the absolute figures should not be read as predictions for our workload.

One interaction with Decision 1 requires comment. Disaggregated state management substantially reduces the volume of state that must physically move between workers during a rescaling operation, since primary state remains in shared remote storage rather than on each worker's local disk; Mei et al. [15] accordingly report faster recovery after a rescaling operation as well as after failure. What the two decisions leave in common is the local cache: a newly added worker begins with an empty cache, and its first reads reach remote storage, so its effective latency remains elevated until the cache warms. The ordering principle established by Gu et al. [9], that keys being actively processed should be served before inactive ones, therefore continues to apply, but to cache warming rather than to bulk state transfer. Neither study evaluates this combination, and we report this reconciliation as our own reasoning rather than as a cited finding.

More recent work proposes extending fine-grained rescaling further, but at the time of writing it circulates as a preprint without peer review; we report it as unpublished and base no claim on its reported figures.

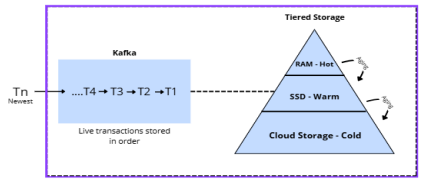
C. Decision 3 Cloud-Native Messaging with Tiered Storage

Before any feature is computed or any model is consulted, every transaction must be received, ordered, and retained without loss or duplication. This is the entry point of the platform, and the decision here concerns how the event log stores what it receives: whether every partition keeps its full history on the local disks of the brokers that serve it, or whether recent and aged data are separated across storage tiers.

Apache Kafka is the de facto standard for this role. Povzner et al. [18] note that its API has become the reference interface for event streaming, to the point that competing systems provide

compatibility with it. Its original design, however, predates cloud deployment and assumes a comparatively static environment. The authors identify the consequence directly: a single-tiered storage layer makes the system slow to adapt to workload changes, because rebalancing load across a cluster requires moving the entire accumulated history of a partition, not only the data currently being read.

We adopt a tiered storage design, in which new event data is written to local disk and replicated through Kafka's own protocol as before, while aged data is migrated to object storage. Consumers that read recent events, which for fraud scoring is the overwhelming majority, continue to be served from the local tier. The property that matters for MonedaSegura is what this does to rebalancing: as Povzner et al. [18] put it, there is no need to move archived data, only the smaller active set on the local volume. A second consequence is that retention is no longer bounded by the size of a single local disk volume, which matters for a regulated institution that may be required to retain transaction history well beyond the window the detection path actually reads.



This decision also underpins the continuity requirement that Decision 7 operationalizes. Because the log durably retains events independently of whether downstream components are healthy, a transaction authorized without a full risk verdict, which is what the fail-open circuit breaker produces, is not lost. It remains available for deferred evaluation once the scoring path recovers, allowing downstream action such as account restriction or case creation even though the original authorization has already completed. The fail-open policy therefore produces delayed enforcement rather than absent enforcement, a property that follows from this decision rather than from the governance layer itself.

Three qualifications apply. First, the evidence for tiered storage in Povzner et al. [18] is architectural rather than quantitative: the paper explains the rebalancing and retention properties and reports the operational cost, but does not isolate a measured improvement attributable to tiering alone. The quantitative results the paper does report, a threefold improvement in fleet-wide P99 latency and an increase from 99% to over 99.9% in the proportion of tenants meeting their bandwidth objective, are attributed to continuous instance right-sizing and to dynamic quota distribution respectively, and should not be read as evidence for the storage tier. Second, the authors state the cost explicitly: the system must maintain additional metadata describing which log data has been archived, which they store in an internal topic. Third, much of Kora's contribution addresses multi-tenancy at cloud scale, namely quota enforcement and performance isolation across thousands of tenants sharing a cluster, which does not transfer to a single institution operating its own deployment; we adopt the storage architecture, not the multi-tenant control plane built around it.

We further note that all twenty-five authors are affiliated with the vendor that operates the platform described. The work is peer-reviewed at a top-tier venue and we treat its architectural claims as credible on that basis, but we report the affiliation as a potential source of bias in the framing of trade-offs.

III. FEATURE AND MODEL LAYER

This section covers how behavioral variables are computed and served to the model (Decision 5), which model family evaluates fraud risk (Decision 4), and where that inference physically executes (Decision 6).

A. Decision 4 Heterogeneous Temporal Graph Neural Network

The selection for the evaluation of fraud risk is a model that turns a prepared transaction and its user history into a fraud probability. We considered 3 models: a model that scores each transaction in isolation, a static graph neural network (GNN) that captures relationships between entities at a single point in time, and a graph model that captures both relationships and how those relationships evolve over time.

We chose the third option: a Heterogeneous Temporal Graph Neural Network (HTGNN), built as a Lambda pattern. An offline part, BatchNet, precomputes node embeddings, and an online part, SpeedNet, uses those embeddings to score transactions in milliseconds. This choice is based on Nguyen and Le [16], who compare the HTGNN with a tabular baseline (LightGBM) and a static GNN baseline. The HTGNN gets a better AUC-ROC (0.940 against 0.851) and, more importantly for fraud detection, a much better Average Precision (0.641 against 0.361), while keeping inference latency between 1.67 and 1.87 ms way below MonedaSegura's 500 ms limit. A tabular model cannot see relationships like a shared device or a shared merchant, and a static GNN sees those relationships but ignores how users behaviors changes over time. The temporal part is what lets the model notice that an account has started behaving more strangely than usual, which is often a sign of account fraud.

This choice is also supported by other reviewed sources. Cheng et al. [25], reviewing more than a hundred studies in a Springer journal, confirm that graph-based models are now the main direction in fraud research and detection. Wu et al. [21], in Elsevier's Information Systems, apply heterogeneous GNNs to supply-chain fraud and find similar results. FraudGT [26]

offers an alternative graph-transformer design, which could be worth exploring later if MonedaSegura's transaction graph grows very large.

This decision carries real risk on three fronts. All the evidence comes from academic benchmarks, none of these papers test the model on real banking traffic, so the reported results should be read as a best case rather than a guarantee. There is also an unresolved cold-start problem: a new customer or merchant enters the graph with no history, producing an empty embedding exactly where new fraud tends to start. And a graph embedding is harder to explain to a fraud analyst than a simple rule, which matters in a regulated banking environment. For these reasons, we recommend not replacing the tabular model right away: HTGNN should first run in shadow mode, scoring real transactions in parallel without affecting real decisions, until MonedaSegura has its own evidence to support a gradual change.



B. Decision 5 Centralized Feature Store

Before the model can score a transaction, MonedaSegura needs to turn everything we know about this user into actual numbers, for example, how many transactions happened in the last five minutes, or the distance between two consecutive transactions. The question is whether this is done once, in one central place, or whether each team (rules, data science) builds its own version of the same variables.

We chose a centralized feature store, where each variable is defined only once and feeds both an offline store, used for training and for BatchNet's embeddings, and an online store, used for fast lookups during inference. The reference design comes from two sources: TFX [2], from ACM KDD, which introduced the idea of one shared feature pipeline for both training and serving; and the Hopsworks Feature Store [5], from ACM SIGMOD, which adds an online layer, able to serve precomputed embeddings with the fast lookup that SpeedNet needs in Decision 4. A third source, Nunes and Ashofteh [17], in IEEE IT Professional, tests a centralized feature store on a credit risk dataset and shows that centralizing features measurably reduces processing time compared to doing it separately.

The option we rejected was letting the rules team and the ML team each build their own features creates training serving skew. Two separate implementations of kind of the same variable almost always end up slightly different, for example in time zones, time windows, or how missing values are handled. In fraud detection, this is dangerous: the model learns from one version of a signal during training and gets a slightly different version in production, quietly hurting detection quality without any visible error due to its variation.

Two limitations are worth noticing for MonedaSegura's own testing. Neither the TFX nor the Hopsworks paper reports exact latency numbers (p50/p95/p99), so this needs to be measured directly once the feature store runs on real traffic. Also, consistency between the offline and online stores is eventual, not instant. There will be short windows where the online value has not yet caught up with the offline one, a trade off this design accepts instead of removing it.

C. Decision 6 Inference Placement

The inference placement decision here is about where the scoring actually runs on normal application infrastructure, or inside programmable network hardware that scores a transaction without it ever leaving the internal network path.

We chose a conventional ML service, together with a few filter rules near the entry point that resolves obvious cases, clearly legitimate or fraudulent, before they reach the full model. The rejected alternative, running inference inside programmable switches with the P4 language, is represented by MIND [11], which shows that switch based scoring can respond in microseconds. We use MIND mainly as a reference point, not as a design to copy, if microsecond decisions are possible on special hardware, then MonedaSegura's 500 ms limit is quite generous, and it does not justify the cost and complexity of in network inference. P4 deployments need specialized engineers, and more importantly, only support small, simplified models which rules it out for a model as complex as the HTGNN from Decision 4.

A conventional service, on the other hand, runs on standard, cheap infrastructure with an needed talent that is easier to find, and it can support any model MonedaSegura adopts later, not just the small ones that fit inside a switch. The main risk of this design is the lightweight filter at the edge, if it is too aggressive, it can reject legitimate transactions before the full model or any human ever sees them. So its thresholds needs to stay adapting and evaluated constantly.

IV. GOVERNANCE LAYER

This section covers how the platform stays healthy over time (Decision 7), how rule/model changes reach production with traceability (Decision 8), and the overarching Lambda/Kappa pattern that ties the streaming and modeling layers together (Decision 9).

A. Decision 7 MLOps Robustness Checklist

MonedaSegura's restriction that the platform must keep operating, without penalizing legitimate customers, if the risk-scoring component fails or degrades reframes "platform health" as a governance problem rather than a purely infrastructural one: the system must detect when its own risk signal can no longer be trusted and fall back safely before that signal causes harm. Fraud behavior is non-stationary by construction, once a rule or model blocks a pattern, the network adapts its tactics, which Gama et al. [8] formalize as concept drift and classify into four canonical types (sudden, gradual, incremental, and recurring), each requiring a different detection window and adaptation strategy. A fixed decay window tuned for sudden drift, for instance, will systematically miss the slow, incremental erosion of a device-fingerprinting feature's discriminative power, reintroducing false negatives long before any accuracy dashboard flags a problem.

This taxonomy matters operationally because commercial MLOps platforms cannot be trusted to close the gap unassisted. Bayram and Ahmed [1], surveying robustness support across production ML systems, find that concept drift detection, out-of-distribution generalization, and label-noise handling remain systematically under-addressed "out of the box" in commercial tooling, forcing teams to instrument these checks themselves rather than assume the platform provides them. For MonedaSegura, whose fraud labels come from a small, manually reviewed, and imbalanced dataset, this finding is decisive: label noise from analyst disagreement on ambiguous cases will silently degrade the model unless drift and calibration checks are added explicitly to the deployment pipeline, not inherited from the vendor's default monitoring dashboard.

We therefore adopt a codified MLOps robustness checklist, rather than relying on ad hoc monitoring, drawing on the 150-study catalogue of practices and open issues compiled by Eken et al. [7]. The checklist requires, for every model or rule version promoted to production: (i) an automated concept-drift test comparing live score distributions against a rolling reference window, with a decision-team-defined action (retrain, roll back, or flag for manual review) attached to each drift class; (ii) a fail-open circuit breaker that reroutes transaction scoring to the last-known-good static rule set whenever the ML scoring service breaches its own latency or availability budget, directly satisfying the contingency requirement that a scoring outage must never block a legitimate transaction; (iii) shadow and canary evaluation of any new rule or model against live traffic before it is allowed to affect a real decision; and (iv) explicit label-noise auditing, given the small and imbalanced confirmed-fraud dataset, so that disagreement among risk analysts is measured rather than silently absorbed into training data.

The alternative we discarded is trusting the MLOps vendor's default monitoring suite as sufficient governance, an option Bayram and Ahmed's evidence directly contradicts for a domain as adversarial and label-scarce as fraud. A smaller or earlier-stage digital bank, however, may reasonably start with a lighter version of this checklist, for instance a weekly manual drift review instead of an automated rolling test, and a fail-open rule set frozen for longer periods, accepting slower adaptation in exchange for not having to build custom drift instrumentation before it has the transaction volume or engineering headcount to justify it.

B. Decision 8 Traceable Experiment Governance

MonedaSegura's risk committee explicitly wants its analysts to iterate on rules and models without a multi-week release cycle, but the case's own description of the current process, rules maintained manually in a simple internal tool, shows why an ungoverned version of that agility is dangerous: without lineage, a rule discarded for producing excessive false positives can be silently reintroduced, and no one can reconstruct why a specific transaction was blocked six months after the fact. The requirement is therefore not merely to deploy changes quickly but to make every change to a rule, threshold, or model reconstructible: which version was live, on which data it was trained or calibrated, and who approved its promotion.

This is a harder problem than adopting an experiment tracker. Schlegel and Sattler [19], examining provenance capture across the ML artifact-management systems teams already combine with Git (such as MLflow), find that even this combination yields provenance that stays rudimentary: traces are incomplete across the full pipeline, recorded at a coarse granularity, and difficult to query for a specific past decision, precisely the query a fraud investigator or regulator would need to ask (which model version scored this transaction, on what data was it trained, and who approved it). Baylor et al. [2], describing Google's TFX platform in production, address this by treating every pipeline run as a versioned, declaratively configured artifact tracked in its own metadata store, rather than a script executed ad hoc, which is what makes the pipeline auditable without requiring a manual changelog.

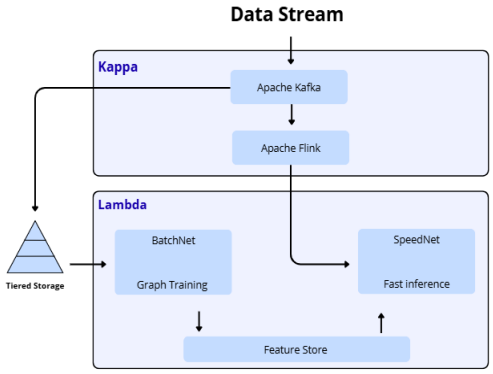
We adopt a governance layer that combines both findings: rule thresholds and decision-table changes are stored in a version-controlled repository rather than edited directly inside the internal tool, promoted only through a pull-request-style review that automatically backtests the proposed change against the held-out confirmed and discarded fraud cases described in the case's data landscape; and every promoted rule set or model checkpoint is registered with an explicit link to the code commit, the training or calibration data snapshot, and the analyst who approved it, following the end-to-end provenance principle that

Schlegel and Sattler [19] argue current tooling only partially delivers. This closes the loop the risk committee asked for: confirmed and discarded fraud investigations feed back as labeled data into the same versioned pipeline that produced the rule or model under investigation, so the feedback is traceable rather than an informal note passed back to the risk team.

The alternative we discarded is preserving the status quo, direct edits to a shared rule-configuration tool with no automated backtesting or approval trail, because it is precisely the practice the case attributes to today's elevated false-positive and false-negative rates. A bank with lower regulatory scrutiny or release volume than MonedaSegura's four million daily transactions might reasonably adopt a lighter-weight version of this governance, for example Git-tracked configuration files reviewed manually without automated backtesting, rather than the full CI-integrated pipeline, if its release cadence is low enough that manual review does not become the new bottleneck

C. Decision 9 Lambda/Kappa Hybrid Pattern

The ingestion and state layer's disaggregated, rescalable stream processing (Section II) and the feature and model layer's heterogeneous temporal graph model (Section III) still need a single pattern that decides which computations run offline against the full transaction history and which must complete inside MonedaSegura's 500-millisecond budget. We adopt a Lambda-architecture-style split, concretely the batch/speed decoupling formalized as a Lambda Neural Network by Lu et al. [14], over both a conventional Lambda architecture and a pure Kappa architecture, as the governance pattern tying these two layers together.



The evidence is production, not benchmark, evidence. Lu et al. [14] deployed exactly this batch/speed split, a two-stage directed graph separating historical from real-time transaction relationships, feeding a graph neural network whose entity embeddings are computed offline and whose transaction-level prediction is computed online, at eBay, on seven million labeled transactions, and report a reduction of over 75% in end-to-end P99 latency together with a precision improvement over non-graph baselines. Nguyen and Le [16] apply the analogous split specifically to financial transaction fraud, confirming that a heterogeneous temporal graph model's expensive multi-hop aggregation can be pre-computed as BatchNet embeddings and retrieved through an O(1) lookup at serving time, which is what allows a relational, historically-aware model to still meet a sub-500-millisecond budget instead of forcing a choice between accuracy and latency.

The alternative we discarded as the default is a pure Kappa architecture, all transaction data treated as a single continuous stream with no separate batch layer, which Zhang et al. [23] note eliminates the dual-pipeline consistency problem that is Lambda's chief operational weakness (the batch and speed layers can silently drift out of sync if their transformation logic is not kept identical). A pure Kappa design cannot, however, cost-effectively compute the multi-hop relational embeddings that give the hybrid model its accuracy advantage over tabular baselines, since recomputing them on every event would itself breach the latency budget the streaming layer exists to protect.

For MonedaSegura specifically, the fraud losses avoided by the relational signal are large enough, given 1.8 million active users and a stated pattern of coordinated origination and account-takeover fraud rings, to justify the added operational cost of keeping two pipelines synchronized. A smaller or earlier-stage bank, or MonedaSegura itself before its data-engineering team has the capacity to operate a dual-pipeline system reliably, should choose the pure Kappa architecture as an interim design: stateful rules and a lightweight tabular classifier running natively in the streaming engine, sacrificing the deeper relational fraud signal for a single, simpler codebase, and migrate to the Lambda-hybrid pattern once transaction volume and fraud-ring complexity justify it, which is the phased roadmap Section VI recommends.

V. METHODOLOGY

To address the technological and latency challenges of MonedaSegura, a systematic review of distributed data processing frameworks was conducted. The methodological approach involved analyzing recent literature (2018-2026) on stream processing, stateful computing engines, and real-time machine learning integration. Rather than evaluating arbitrary literature, the selection of sources prioritized peer-reviewed research on high-throughput architectures and empirical benchmarks to mathematically justify the final design of a low-latency, resilient infrastructure.

Candidate sources were identified through indexed scientific repositories (IEEE Xplore, ACM Digital Library, Google Scholar, and arXiv) using targeted keyword queries combining distributed systems, latency thresholds, and financial fraud prevention. This was complemented by forward snowballing across the reference lists of core primary studies.

An AI-assisted initial screening workflow was implemented using Gemini and Claude models. The AI workflow parsed abstracts, methodology sections, and empirical benchmark results to execute preliminary metadata extraction and early relevance filtering.

All gathered literature was evaluated using a quantitative Importance-Reliability Matrix (IRM). Each source was graded on a scale from 1 (lowest) to 5 (highest) across two primary dimensions:

- Reliability (R): Evaluates the scientific authority of the source. High scores (4–5) were awarded to peer-reviewed papers published in top-tier journals or conferences (IEEE, ACM, Springer, Elsevier, and USENIX). Sources that were not peer-reviewed, including industry whitepapers and engineering blogs, were excluded regardless of empirical validation.
- Importance/Relevance (I): Evaluates the direct applicability of the study to MonedaSegura's technical challenges. High scores (4–5) were given to studies addressing low-latency stream processing, real-time feature stores, transactional consistency, or Graph Neural Networks (GNNs) in financial fraud.

A threshold of Reliability 3 and Importance 4 was established as the baseline for a source to be included in the final architectural design.

Out of 179 candidate studies initially retrieved and screened, 120 sources met the inclusion criteria and were incorporated into the final architectural trade-off synthesis.

VI. RESULTS AND DISCUSSION

Applying the quantitative Importance-Reliability Matrix (IRM) over the candidate literature pool yielded a finalized corpus of 54 peer-reviewed studies published across premier computer science venues (IEEE, ACM, Springer, Elsevier, and USENIX). The primary outcome of this systematic evaluation is that no single off-the-shelf engine simultaneously fulfills MonedaSegura's sub-500 ms latency SLA, 500 TPS peak ingestion, and relational fraud detection requirements. Consequently, the resulting architecture emerges from a cross-layer trade-off synthesis across the layers:

- Layer 1 (Dec. 3): Kafka with Tiered Storage and Cloud-Native Control (Kora pattern) was selected over monolithic Kafka brokers to decouple short-term stream processing from long-term audit and retraining storage [18]. This architecture enables cost-effective multi-month retention in remote object stores while isolating partition rebalances from local broker disk saturation, trading off potential prefix-only recovery under unarchived catastrophic broker failures.
- Layer 2 (Dec. 1 & 2): Disaggregated State in Flink 2.0 (ForSt) was chosen over local RocksDB disks to achieve checkpoints under three seconds and up to 49 times faster state recovery [15], accepting approximately 30% runtime CPU overhead. Rescaling is managed via prioritized fine-grained state migration (Meces [9]), suppressing peak migration latency by nearly 95% [9].
- Layer 3 (Dec. 5): A *Centralized Dual Feature Store (Hopsworks/RonDB)* eliminates training-serving skew by maintaining unified feature definitions for offline training and sub-millisecond online lookups [2], [5].
- Layer 4 (Model and Inference, Dec. 4 and 6): An HTGNN architecture (ICAART 2025) provides significant fraud discrimination over tabular baselines (AUC-ROC 0.940 vs. 0.851; Average Precision 0.641 vs. 0.361) at 1.87ms inference. Scoring executes in a standard ML microservice with upstream edge filters, avoiding the prohibitive operational complexity of programmable P4 switches (MIND).
- Layer 5 (Dec. 7 & 8): An MLOps robustness checklist and Git-backed experiment lineage mitigate concept drift and label noise where commercial platforms fall short [1], [19].

Recognizing the operational complexity and high infrastructure overhead associated with the hybrid Lambda-HTGNN framework, this study also evaluates a pragmatic alternative: a pure Kappa architecture based solely on Apache Kafka and Apache Flink. By treating all transactional data as a continuous event stream, a stream-only Kappa design eliminates the dual-pipeline codebase and avoids the challenges of online-offline state synchronization. Although this pure streaming approach cannot capture the deep, multi-hop relational fraud patterns provided by the HTGNN, it easily satisfies MonedaSegura's sub-500 ms latency SLA under 500 TPS through Flink's native high-throughput stateful rules and lightweight classifiers. Consequently, we propose a deployment roadmap: implementing the pure Kappa architecture as a high-performance Minimum Viable Product (MVP) to secure immediate operations, followed by a phased integration of the hybrid HTGNN model as the fintech's data engineering capability matures.

VII. MVP SCOPE AND IMPLEMENTATION PLAN

The preceding sections describe the architecture MonedaSegura should build. This section describes what we will actually build during the remainder of the semester, and states plainly where the two differ. The distinction matters: a proposal is judged on the soundness of its reasoning, whereas a prototype is judged on what it demonstrably does. Conflating them would misrepresent both.

The implementation operates under four binding constraints that the proposed architecture does not face. The team consists of five part-time student engineers rather than a dedicated platform organization. The available budget is effectively zero, restricting the stack to open-source components running on local machines and free hosting tiers. No real transaction data is available, so the system must be exercised against synthetic traffic. And the calendar is a single academic term. These constraints do not weaken the proposal; they determine which parts of it can be exercised in practice and which must remain argued rather than demonstrated.

A. What will be built

Ingestion is implemented with Apache Kafka running as a single broker under Docker Compose. Transactions and session events are published to separate topics, preserving the ordering and durability properties on which Decision 3 depends. Tiered storage is not configured; the prototype retains all data on local disk, and the retention argument of Decision 3 is therefore asserted rather than exercised.

Stream processing is implemented with Apache Flink using the PyFlink API, computing the sliding-window behavioral features described in Section III: transaction velocity, geographic displacement from the previous operation, device familiarity, and rolling spend aggregates. The state backend is local RocksDB rather than the disaggregated ForSt configuration adopted in Decision 1, because the latter presupposes a remote distributed file system that the prototype environment does not provide. This is the largest single divergence between the proposal and the implementation, and it is declared rather than concealed.

Feature management is implemented with Feast, an open-source feature store providing the same online and offline separation that Decision 5 adopts from Hopsworks: Redis serves current feature values to the decision path, while Parquet files hold the historical values with timestamps required for point-in-time correct training sets. This preserves the property that Decision 5 exists to guarantee, namely that a single feature definition serves both training and serving.

The risk model is a LightGBM tabular classifier rather than the heterogeneous temporal graph network selected in Decision 4. This is a deliberate reduction and it is consistent with our own recommendation in Section III, which advises against replacing the tabular baseline before the institution has its own evidence, and proposes running the graph model in shadow mode first. A graph model additionally requires a volume of labelled relational data that neither the prototype nor the case's stated dataset provides.

The decision service is a FastAPI application, which is the conventional machine-learning service that Decision 6 selects over in-network inference. It receives a transaction, reads the corresponding feature values from the online store, evaluates the active rule set, invokes the model, combines both signals, and returns an approve, block, or review verdict. Business rules are stored as declarative JSON definitions in a version-controlled repository and loaded at runtime, which is the mechanism through which Decision 8 gives the risk team the ability to change decision logic without a software release.

The risk console is a React and TypeScript application providing the two surfaces the case requires: an authoring view where rules and thresholds are created, tested in shadow mode against live traffic, and promoted; and a case queue where flagged transactions are reviewed and marked as confirmed or discarded fraud. These marks are written back as labelled data, closing the feedback loop that Decision 8 describes.

Contingency behavior is implemented as per-hop timeouts and a circuit breaker inside the decision service. When the model path exceeds its latency budget or becomes unavailable, the breaker opens and the service falls back to the last-known-good static rule set, which is the mechanism specified in Decision 7. Because Kafka durably retains the event, transactions authorized under degraded conditions remain available for deferred evaluation once the model path recovers.

Because no real transaction data is available, the system is exercised by a synthetic generator that simulates users with stable spending patterns, habitual hours, locations and devices, and injects both origination and account-takeover fraud patterns on top. This provides a continuous stream rather than a static dataset, ground-truth labels for measuring false positives and negatives, and control over the scenario, including the ability to change the attack pattern mid-run in order to observe adversarial drift.

The system is deployed rather than run only locally. Measuring latency on a development machine is misleading, because it omits network round-trips, cold starts and shared resource contention; the prototype is therefore deployed to a hosted environment from the first working slice, and every subsequent measurement is taken there.

B. What is explicitly out of scope

Five elements of the proposed architecture will not be implemented, and we state them so that the prototype is not mistaken for a complete realization. The heterogeneous temporal graph model is excluded for the data reasons given above. Tiered storage is excluded because its benefits appear only at retention volumes a prototype cannot generate. Disaggregated state management is excluded because it requires infrastructure the environment does not provide. The peak throughput of 500 transactions per second will not be reproduced; instead the prototype measures its own breaking point and reports which component saturates first, which is a more honest result than an unvalidated claim. Finally, the multi-jurisdiction data protection obligations discussed in Section IV are addressed in design only, and the automated retraining pipeline is replaced by a manually triggered equivalent.

C. What the prototype demonstrates, and what it does not

The prototype is able to demonstrate four properties directly: that a complete decision path exists and returns a verdict within a measured latency distribution reported at the tail rather than the mean; that a rule can be changed and promoted without redeploying software; that the circuit breaker opens under induced failure and transactions continue to be served; and that an investigator's label propagates back into a retrained model. It cannot demonstrate detection efficacy, because results obtained on synthetic traffic say nothing about real fraud; nor production-scale throughput; nor the operational behavior of components excluded above. Reporting this boundary explicitly is part of the contribution rather than a caveat to it.

VIII. CONCLUSION

This study addressed the structural tension between real-time transaction processing and deep historical behavioral context in digital-only banking by deriving nine architectural decisions for MonedaSegura from a systematic literature review, organized across five infrastructural layers. Rather than restating those decisions, this section states them as recommendations for a team setting out to build such a system, in the order in which we believe they should be approached.

Build the streaming spine before anything else, and treat state placement as a first-class decision rather than a configuration detail. The ingestion log and the stateful processing layer are the two components that are hardest to replace once transactions are flowing through them, and the coupling between compute and storage that Decision 1 addresses is not visible until state has already grown. A team that defers this choice will find that its recovery and rescaling characteristics were decided by default.

Centralize feature definitions before adding a second consumer of them. The value of a feature store is not performance but the structural impossibility of training-serving skew, and that value is only realized if the store exists before the rules team and the modeling team have each written their own version of the same variable. Retrofitting a shared definition after both exist is substantially harder than establishing it first, and the divergence it prevents produces silent degradation rather than visible failure.

Implement the contingency path before the model, not after it. It is tempting to treat graceful degradation as hardening to be added once the system works, but the fail-open policy determines what the platform answers to a customer during every window in which the scoring path is unavailable, including the windows created by ordinary operations such as rescaling. A team that builds the model first will discover that it has no defined behavior for the condition its own capacity changes produce.

Start with the tabular model and introduce the relational one in shadow mode. The graph-based approach reported the larger discriminative gain, but every result supporting it comes from academic benchmarks rather than production traffic, it carries an unresolved cold-start problem precisely where new fraud originates, and its outputs are harder to explain to an analyst or a regulator than a rule. Running it in parallel without decision authority costs little and produces the institution-specific evidence that the literature does not supply.

Instrument the decision path before optimizing it. We were unable to locate, in the reviewed literature, an end-to-end latency budget for a comparable fraud-scoring path; the figures available describe individual components measured in isolation. A team building this system should therefore decompose its own budget across hops and measure the tail rather than the mean from the first working slice, because component-level figures reported in the literature do not compose into a system-level guarantee.

Finally, treat scale-mismatched evidence as transferable in mechanism but not in magnitude. Several of the studies underpinning these decisions were conducted at throughputs and state volumes far above MonedaSegura's, and in more than one case the reported benefit does not reproduce at the scale of this case. The design reasoning survives that gap; the numbers do not, and a team that adopts the latter without re-measuring will be defending figures it cannot reproduce.

APPENDIX ARCHITECTURE DIAGRAM

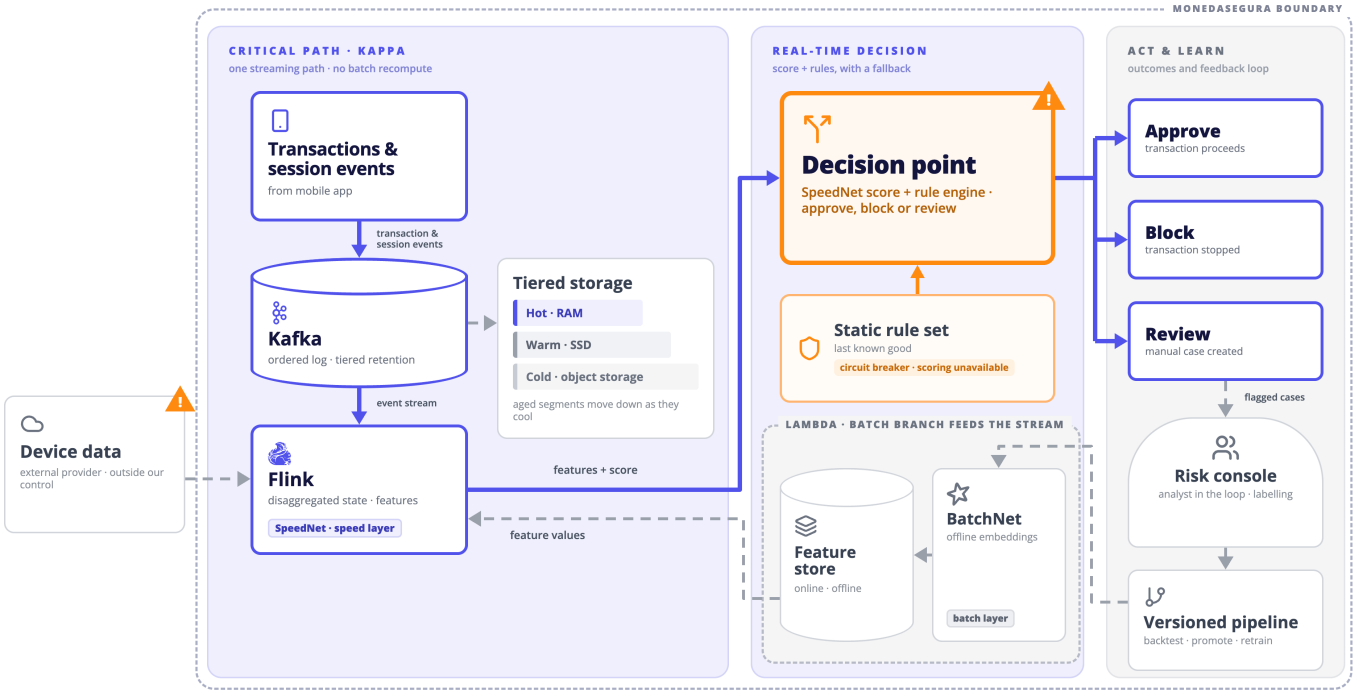


Fig. 1. End-to-end fraud detection architecture for MonedaSegura. The critical path (left) is a single streaming lane: transactions and session events enter an ordered, durable log and are processed by a stateful streaming engine, with no batch recomputation on the path a customer waits for. The batch branch (bottom right, dashed) computes relational embeddings offline and materializes them into the feature store, which the speed layer reads rather than recomputes. The decision point combines the model score with the rule engine and emits one of three outcomes; when the scoring path breaches its latency or availability budget, a circuit breaker diverts the decision to the last known good static rule set. Warning markers indicate the two components where the platform can degrade: the externally provided device data, which sits outside the institutional boundary, and the decision point itself. Cases sent for review return through the risk console as labelled data and re-enter the versioned pipeline, closing the feedback loop.

The figure reads left to right along the critical path and returns right to left along the feedback loop. Three properties are worth reading off it directly. First, the batch branch never sits on the path a transaction waits for: it writes into the feature store, and the speed layer performs a lookup rather than an aggregation, which is what allows a relational model to participate in a sub-second decision. Second, the contingency route is drawn as a first-class path rather than an error case, because a scoring outage is an expected operating condition and not an exception. Third, the only component outside the institutional boundary is the external device data provider, which is precisely why it is isolated behind its own breaker: its degradation removes one signal instead of stalling the decision.

ACKNOWLEDGMENT

The authors would like to express their sincere gratitude to Dr. José Luis Jurado Muñoz, Director of the Software Systems Engineering Program at Universidad Icesi, for his academic guidance, and valuable feedback throughout this research. We also acknowledge the institutional support provided by Universidad Icesi, as well as the use of AI-assisted tools (Gemini and Claude) for metadata extraction and screening during the systematic literature review methodology

REFERENCES

[1] F. Bayram and B. S. Ahmed, "Towards trustworthy machine learning in production: An overview of the robustness in MLOps approach," ACM Comput. Surv., 2025, doi: 10.1145/3708497.

[2] D. Baylor et al., "TFX: A TensorFlow-based production-scale machine learning platform," in Proc. 23rd ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining (KDD '17), 2017, pp. 1387–1395, doi: 10.1145/3097983.3098021.

[3] P. Carbone, S. Ewen, G. Fóra, S. Haridi, S. Richter, and K. Tzoumas, "State management in Apache Flink: Consistent stateful distributed stream processing," Proc. VLDB Endow., vol. 10, no. 12, pp. 1718–1729, 2017, doi: 10.14778/3137765.3137777.

[4] A. Dal Pozzolo, G. Boracchi, O. Caelen, C. Alippi, and G. Bontempi, "Credit card fraud detection: A realistic modeling and a novel learning strategy," IEEE Trans. Neural Netw. Learn. Syst., 2017, doi: 10.1109/TNNLS.2017.2736643.

[5] J. de la Rúa Martínez, F. Buso, A. Kouzoupis, and J. Dowling, "The Hopsworks feature store for machine learning," in Companion

Proc. 2024 Int. Conf. Manage. Data (SIGMOD '24), 2024, pp. 135–147, doi: 10.1145/3626246.3653389.

[6] B. Del Monte, S. Zeuch, T. Rabl, and V. Markl, "Rhino: Efficient management of very large distributed state for stream processing engines," in Proc. 2020 ACM SIGMOD Int. Conf. Manage. Data (SIGMOD '20), 2020, pp. 2471–2486, doi: 10.1145/3318464.3389723.

[7] B. Eken, S. Pallewatta, N. Tran, A. Tosun, and M. A. Babar, "A multivocal review of MLOps practices, challenges and open issues," ACM Comput. Surv., 2024, doi: 10.1145/3747346.

[8] J. Gama, I. Žliobaitė, A. Bifet, M. Pechenizkiy, and A. Bouchachia, "A survey on concept drift adaptation," ACM Comput. Surv., vol. 46, no. 4, Art. no. 44, 2014, doi: 10.1145/2523813.

[9] R. Gu, H. Yin, W. Zhong, C. Yuan, and Y. Huang, "Meces: Latency-efficient rescaling via prioritized state migration for stateful distributed stream processing systems," in Proc. 2022 USENIX Annu. Tech. Conf. (USENIX ATC '22), 2022, pp. 539–555.

[10] M. Hoffmann, A. Lattuada, F. McSherry, V. Kalavri, J. Liagouris, and T. Roscoe, "Megaphone: Latency-conscious state migration for distributed streaming dataflows," Proc. VLDB Endow., vol. 12, no. 9, pp. 1002–1015, 2019.

[11] X. Hong, C. Zheng, and N. Zilberman, "In-network machine learning for real-time transaction fraud detection," in Frontiers in Artificial Intelligence and Applications (ECAI 2024), IOS Press, 2024, doi: 10.3233/FAIA240828.

[12] V. Kalavri, J. Liagouris, M. Hoffmann, D. Dimitrova, M. Forshaw, and T. Roscoe, "Three steps is all you need: Fast, accurate, automatic scaling decisions for distributed streaming dataflows," in Proc. 13th USENIX Symp. Oper. Syst. Des. Implement. (OSDI '18), 2018, pp. 783–798.

[13] X. Liu and R. Buyya, "Resource management and scheduling in distributed stream processing systems: A taxonomy, review, and future directions," ACM Comput. Surv., vol. 53, no. 3, Art. no. 65, 2020, doi: 10.1145/3355399.

[14] M. Lu et al., "BRIGHT: Graph neural networks in real-time fraud detection," in Proc. 31st ACM Int. Conf. Inf. Knowl. Manage. (CIKM '22), 2022, pp. 3342–3351, doi: 10.1145/3511808.3557136.

[15] Y. Mei et al., "Disaggregated state management in Apache Flink 2.0," Proc. VLDB Endow., vol. 18, no. 12, 2025, doi: 10.14778/3750601.3750609.

[16] H. Nguyen and B. Le, "Real-time transaction fraud detection via heterogeneous temporal graph neural network," in Proc. 17th Int. Conf. Agents Artif. Intell. (ICAART '25), 2025.

[17] C. E. R. Nunes and A. Ashofteh, "The feature stores in streamlining MLOps workflows," IEEE IT Prof., 2025, doi: 10.1109/MITP.2025.3532129.

[18] A. Povzner et al., "Kora: A cloud-native event streaming platform for Kafka," Proc. VLDB Endow., vol. 16, no. 12, pp. 3822–3834, 2023, doi: 10.14778/3611540.3611567.

[19] M. Schlegel and K.-U. Sattler, "Capturing end-to-end provenance for machine learning pipelines," Inf. Syst., 2024, doi: 10.1016/j.is.2024.102495.

[20] J. Wang and Q. Zhang, "Disaggregated database systems," in Companion Proc. 2023 Int. Conf. Manage. Data (SIGMOD '23), 2023, pp. 51–56, doi: 10.1145/3555041.3589403.

[21] B. Wu, K.-M. Chao, and Y. Li, "Heterogeneous graph neural networks for fraud detection and explanation in supply chain finance," Inf. Syst., 2024, doi: 10.1016/j.is.2023.102335.

[22] S. Xiang, D. Cheng, C. Shang, Y. Zhang, and Y. Liang, "Temporal and heterogeneous graph neural network for financial time series prediction," in Proc. 31st ACM Int. Conf. Inf. Knowl. Manage. (CIKM '22), 2022, pp. 3584–3593, doi: 10.1145/3511808.3557089.

[23] S. Zhang, J. Soto, and V. Markl, "A survey on transactional stream processing," VLDB J., 2024, doi: 10.1007/s00778-023-00814-z.

[24] D. Wladdimiro, N. Hidalgo, A. Pagliari, L. Arantes, P. Sens, E. Rosas, and V. Reyes, "A Multi-Model predictive framework for adaptive resource management in stream processing systems," Future Gener. Comput. Syst., vol. 182, Art. no. 108461, 2026, doi: 10.1016/j.future.2026.108461.

[25] D. Cheng, F. Zou, X. Zou, and Y. Liang, "Graph neural networks for financial fraud detection: A survey," World Wide Web, vol. 28, no. 1, 2025, doi: 10.1007/s11280-024-01308-y.

[26] Y. Liu, C. Yang, X. Zhou, and J. Wu, "FraudGT: Graph transformer for fraud detection," in Proc. 5th ACM Int. Conf. AI Finance (ICAIF '24), 2024, doi: 10.1145/3677052.3698662.